ORACLE DATABASE MAINTENANCE AND MONITORING

SECTION 1: AUTOMATING ROUTINE MAINTENANCE TASKS

Trainingcred
Empower. Innovate. Advance.

Introduction to Database Maintenance Automation

Why Automate?

- Ensures consistency and reliability
- Reduces human error and oversight
- Frees up DBA time for strategic tasks
- Enables 24/7 maintenance operations
- Provides audit trails and compliance

Key Benefits

- Predictable maintenance windows
- Reduced downtime
- Improved database performance
- Better resource utilization

Trainingcred
Empower. Innovate. Advance.

Types of Routine Maintenance Tasks

Daily Tasks

- Backup verification
- Alert log monitoring
- Space usage checks
- Performance metric review

Weekly Tasks

- Statistics collection
- Index maintenance
- Backup testing
- Security audit review

Monthly Tasks

- Patch assessment
- Capacity planning review
- Performance trending analysis

Trainingcred

Oracle Scheduler Overview

What is Oracle Scheduler?

- Built-in job scheduling framework
- Replaces legacy DBMS_JOB package
- Enterprise-level scheduling capabilities
- Integrated with Oracle Database

Core Components

- Jobs: Individual tasks to execute
- Programs: Reusable job definitions
- Schedules: Time specifications
- Job Classes: Resource management groups
- Windows: Time periods for maintenance

Trainingcred
Empower. Innovate. Advance.

Creating Your First Scheduled Job

Basic Job Creation Steps

1. Define what needs to run (program/action)
2. Specify when it should run (schedule)
3. Configure job parameters
4. Enable the job
5. Monitor execution

Simple Example Structure

- Job Name: Unique identifier
- Job Type: PL/SQL block, stored procedure, or external script
- Job Action: Code or program to execute
- Schedule: Time-based or event-based
- Enabled: TRUE to activate

Trainingcred

DBMS_SCHEDULER Package

Key Procedures

- CREATE_JOB: Define new scheduled tasks
- CREATE_SCHEDULE: Define reusable schedules
- CREATE_PROGRAM: Create reusable program definitions
- ENABLE/DISABLE: Control job execution
- DROP_JOB: Remove jobs
- RUN_JOB: Execute immediately

Management Procedures

- SET_ATTRIBUTE: Modify job properties
- STOP_JOB: Terminate running job
- COPY_JOB: Duplicate existing jobs

Trainingcred
Empower. Innovate. Advance.

Schedule Types and Expressions

Repeat Intervals

- FREQ=DAILY: Every day
- FREQ=WEEKLY; BYDAY=MON,WED,FRI: Specific weekdays
- FREQ=MONTHLY; BYMONTHDAY=1: First day of month
- FREQ=HOURLY; INTERVAL=4: Every 4 hours

Calendar Expressions

- Support complex scheduling patterns
- Use Oracle's calendaring syntax
- Combine multiple conditions
- Account for holidays and exceptions

Named Schedules

- Create once, use many times
- Easy to modify centrally
- Consistent timing across jobs

Job Chains and Dependencies

What are Job Chains?

- Link multiple jobs in sequence
- Define conditional execution paths
- Handle complex workflows
- Manage dependencies between tasks

Chain Steps

- Each step represents a task
- Steps connected by rules
- Rules evaluate conditions
- Next step based on outcome

Use Cases

- ETL processes
- Backup workflows
- Month-end processing
- Multi-stage data loading

Trainingcred
Innovate Advance

Automating Statistics Collection

Why Statistics Matter

- Optimizer relies on accurate statistics
- Outdated stats lead to poor execution plans
- Automatic collection ensures freshness

Auto Stats Collection

- Runs during maintenance window
- Collects schema and table statistics
- Updates stale statistics automatically
- Configurable through DBMS_STATS

Manual Configuration

- Set specific collection preferences
- Control sampling percentages
- Define collection schedules
- Exclude specific objects

Automating Backup Operations

RMAN Automation Benefits

- Consistent backup schedules
- Automated retention policies
- Built-in error handling
- Integration with media managers

Backup Types to Automate

- Full database backups (weekly)
- Incremental backups (daily)
- Archive log backups (hourly)
- Control file backups (after changes)

Scheduling Strategy

- Level 0: Full backup weekly
- Level 1: Incremental daily
- Archive logs: Multiple times daily
- Validation: Weekly backup checks

Space Management Automation

Automated Space Monitoring

- Tablespace usage thresholds
- Alert when 85% full (warning)
- Alert when 95% full (critical)
- Automatic datafile extension

Proactive Management

- Schedule space usage reports
- Automate segment shrinking
- Purge old data automatically
- Archive historical information

Space Reclamation

- Rebuild fragmented indexes
- Shrink segments with excessive free space
- Drop unused indexes
- Compress historical data

Purge and Archive Operations

What to Purge Automatically

- Audit trail records (older than retention period)
- Recyclebin objects
- Expired backup sets
- Obsolete archive logs
- Temp files and segments

Archive Strategies

- Move old data to archive tables
- Partition by date for easy purging
- Compress archived data
- Offload to cheaper storage

Retention Policies

- Define based on compliance requirements
- Balance storage costs with needs
- Document retention decisions
- Automate enforcement

Health Check Automation

Automated Diagnostics

- Check database availability
- Monitor alert log for errors
- Verify backup completion
- Test recovery scenarios
- Validate data integrity

Health Check Scripts

- Tablespace usage reports
- Invalid object detection
- Lock contention analysis
- Performance bottleneck identification
- Security vulnerability scans

Notification Integration

- Email alerts for failures
- Dashboard updates
- Ticket system integration
- Escalation procedures

Trainingcred

Resource Management with Job Classes

What are Job Classes?

- Group jobs with similar characteristics
- Assign resource consumer groups
- Control resource allocation
- Prioritize critical jobs

Resource Allocation

- Database Resource Manager integration
- CPU allocation percentages
- I/O priority levels
- Parallel execution limits

Creating Job Classes

- Define resource group
- Set logging level
- Configure service affinity
- Assign jobs to class

Trainingcred
Innovative Advance

Monitoring Job Execution

Key Views for Monitoring

- DBA_SCHEDULER_JOBS: Job definitions
- DBA_SCHEDULER_JOB_RUN_DETAILS: Execution history
- DBA_SCHEDULER_RUNNING_JOBS: Currently executing
- DBA_SCHEDULER_JOB_LOG: Job events and changes

What to Monitor

- Job completion status
- Execution duration trends
- Failure rates and patterns
- Resource consumption
- Schedule adherence

Setting Up Alerts

- Job failure notifications
- Long-running job alerts
- Missed execution warnings

Error Handling and Notifications

Built-in Error Handling

- Automatic retry on failure
- Configurable retry count and interval
- Logging of all errors
- State tracking

Notification Methods

- Email notifications (DBMS_SCHEDULER)
- PL/SQL procedures for custom alerts
- Integration with monitoring tools
- SMS or paging systems

Error Response Actions

- Log detailed error information
- Attempt automatic recovery
- Notify appropriate personnel
- Create incident tickets
- Document for analysis

Maintenance Windows

Purpose of Maintenance Windows

- Define preferred maintenance periods
- Automatically enable/disable jobs
- Adjust resource allocation
- Coordinate related activities

Predefined Windows

- MONDAY_WINDOW through SUNDAY_WINDOW
- WEEKNIGHT_WINDOW (Mon-Fri nights)
- WEEKEND_WINDOW (Sat-Sun)

Custom Windows

- Create organization-specific windows
- Define based on business cycles
- Account for time zones
- Handle special events

Window Groups

- Combine multiple windows
- Assign jobs to window groups
- Flexible scheduling across time periods

Best Practices for Job Scheduling

Design Principles

- Keep jobs simple and focused
- Use programs for reusability
- Implement proper error handling
- Document job purposes
- Test in non-production first

Scheduling Guidelines

- Avoid overlapping resource-intensive jobs
- Stagger similar jobs
- Allow buffer time between dependent jobs
- Consider backup windows
- Plan for holidays and maintenance

Maintenance Considerations

- Regular review of job effectiveness
- Archive old job logs
- Update job documentation
- Monitor for schedule conflicts
- Optimize job execution times

External Job Integration

Running External Programs

- Shell scripts
- Operating system commands
- Backup utilities
- Third-party tools

Configuration Requirements

- Create credential objects
- Define external job type
- Specify executable path
- Set environment variables
- Handle return codes

Security Considerations

- Use least privilege credentials
- Validate input parameters
- Secure credential storage
- Audit external job execution
- Restrict file system access

Advanced Scheduling Techniques

Event-Based Scheduling

- Trigger jobs based on database events
- File arrival detection
- Queue message processing
- Application-generated events

Lightweight Jobs

- Quick, frequent tasks
- Minimal overhead
- In-memory execution
- No persistent state

Remote Job Execution

- Execute jobs on remote databases
- Coordinate distributed tasks
- Database link integration
- Cross-platform support

Troubleshooting Scheduler Issues

Common Problems

- Jobs not running on schedule
- Jobs failing silently
- Resource contention
- Unexpected job behavior

Diagnostic Steps

1. Check job state and status
2. Review execution logs
3. Verify schedules and windows
4. Check resource availability
5. Examine error messages

Resolution Strategies

- Adjust job priorities
- Modify resource allocations
- Fix schedule conflicts
- Update job definitions
- Implement better error handling

Trainingcred

SECTION 2: USING ORACLE ENTERPRISE MANAGER FOR DATABASE MONITORING

Introduction to Oracle Enterprise Manager

What is OEM?

- Comprehensive database management platform
- Web-based graphical interface
- Centralized monitoring and administration
- Supports single or multiple databases

Key Capabilities

- Performance monitoring and diagnostics
- Configuration management
- Job scheduling and automation
- Patch and update management
- Comprehensive reporting

Deployment Options

- OEM Cloud Control: Enterprise solution
- OEM Database Express: Single database
- OEM Database Control: Legacy option

OEM Architecture Overview

Components

- OEM Repository: Central data store
- Management Agents: Deployed on targets
- Oracle Management Service (OMS): Middle tier
- Web Console: User interface

Communication Flow

- Agents collect metrics from targets
- Data sent to OMS
- OMS stores data in repository
- Console accesses data via OMS

Scalability

- Monitor thousands of databases
- Hierarchical organization
- Load balancing across OMS instances
- Distributed agent deployment

Accessing OEM Database Express

Connection Methods

- HTTPS://<hostname>:5500/em
- Port configured during installation
- Browser-based access
- Single database focus

Authentication

- Database user credentials
- SYSDBA or SYSOPER privileges for admin tasks
- Regular users for monitoring only
- Integration with enterprise directories

Initial Setup

- Verify EM Express is configured
- Enable HTTPS listener
- Set up user access
- Configure preferences

OEM Home Page Dashboard

Dashboard Sections

- Status summary (up/down/alerts)
- Performance metrics overview
- Space usage indicators
- Recent alerts and incidents
- Resource utilization graphs

Key Metrics Displayed

- CPU utilization percentage
- Active sessions count
- Database throughput (transactions/sec)
- I/O operations per second
- Memory usage statistics

Customization

- Add/remove dashboard widgets
- Arrange layout preferences
- Set refresh intervals
- Create custom views

Performance Hub

Real-Time SQL Monitoring

- Track currently executing SQL
- View execution plans in progress
- Identify resource-intensive queries
- Monitor parallel execution

Active Session History (ASH)

- Sample active sessions every second
- Historical performance analysis
- Wait event identification
- Workload profiling

SQL Tuning Advisor Integration

- Identify poorly performing SQL
- Generate tuning recommendations
- Test alternative execution plans
- Implement optimizations

Monitoring Database Performance

CPU Metrics

- CPU utilization by database
- Top CPU-consuming sessions
- CPU wait time analysis
- Host CPU statistics

Memory Metrics

- SGA and PGA usage
- Buffer cache hit ratio
- Shared pool efficiency
- Memory allocation trends

I/O Metrics

- Disk read/write rates
- I/O wait times
- Tablespace I/O statistics
- Top I/O consumers

Network Metrics

- Network bytes sent/received
- Connection counts
- TNS activity

Wait Event Analysis

What are Wait Events?

- Times when sessions wait for resources
- Indicate performance bottlenecks
- Categorized by wait class
- Critical for performance tuning

Common Wait Classes

- User I/O: Disk read operations
- System I/O: Background process I/O
- Concurrency: Lock and latch waits
- Network: Data transfer waits
- Configuration: Suboptimal settings

Analyzing Waits

- Identify top wait events
- Determine affected sessions
- Find root causes
- Implement solutions

Automatic Workload Repository (AWR)

What is AWR?

- Collects database performance statistics
- Snapshots taken at regular intervals (default: 60 minutes)
- Retains data for 8 days (default)
- Foundation for performance analysis

AWR Reports

- Time period performance summary
- Top SQL statements
- Wait event statistics
- Load profile analysis
- Instance efficiency metrics

Generating Reports

- Specify begin and end snapshot IDs
- Choose report format (HTML/text)
- Select comparison periods
- Schedule automated generation

Automatic Database Diagnostic Monitor (ADDM)

ADDM Overview

- Analyzes AWR data automatically
- Identifies performance problems
- Provides actionable recommendations
- Runs after each AWR snapshot

ADDM Findings

- Performance impact quantified
- Root cause analysis
- Prioritized recommendations
- Expected benefit estimates

Recommendation Types

- SQL tuning opportunities
- Database configuration changes
- Schema modifications
- Hardware/resource upgrades

Implementing Recommendations

- Review findings carefully
- Test in non-production
- Document changes
- Measure improvements

SQL Performance Monitoring

SQL Monitoring Features

- Automatic for long-running queries
- Manual monitoring for specific SQL
- Real-time execution statistics
- Visual execution plan display

Key Information Displayed

- Elapsed time and CPU time
- I/O and memory usage
- Parallel execution details
- Execution plan operations
- Wait events by plan line

Identifying Problem SQL

- High elapsed time
- Excessive buffer gets
- Large disk reads
- Inefficient execution plans
- Missing or unused indexes

Trainingcred
Innovate Advance

Metric Baselines

Purpose of Baselines

- Establish normal performance levels
- Detect anomalies and deviations
- Compare current vs. baseline
- Adaptive thresholds

Types of Baselines

- Static: Fixed threshold values
- Percentage: Relative to baseline
- Statistical: Based on standard deviation
- Template: Applied to multiple targets

Creating Baselines

- Select representative time periods
- Capture during normal operations
- Avoid peak or off-peak extremes
- Update periodically

Using Baselines

- Compare current performance
- Identify degradation trends
- Set intelligent alert thresholds
- Support capacity planning

Alert and Incident Management

Alert Types

- Metric alerts: Threshold violations
- Target availability alerts
- Policy violations
- Job failures
- Custom defined alerts

Alert Severity Levels

- Critical: Immediate attention required
- Warning: Potential issue developing
- Clear: Alert condition resolved

Incident Creation

- Automatic from critical alerts
- Manual incident creation
- Associated diagnostic data
- Incident lifecycle tracking

Incident Resolution

- Problem diagnosis
- Solution implementation
- Incident closure
- Root cause documentation

Configuring Alert Thresholds

Threshold Types

- Warning: Early notification level
- Critical: Urgent action required
- Clear: Returned to normal

Commonly Monitored Metrics

- Tablespace space used (%)
- CPU utilization (%)
- Active sessions count
- Response time (seconds)
- Archive log generation rate

Setting Effective Thresholds

- Base on historical data
- Account for growth trends
- Set warning before critical
- Avoid alert fatigue
- Adjust seasonally

Notification Rules and Methods

Notification Methods

- Email: Most common method
- SNMP traps: Integration with monitoring tools
- PL/SQL procedures: Custom actions
- OS scripts: System-level responses

Rule Configuration

- Define conditions that trigger
- Select notification method
- Specify recipients
- Set notification frequency
- Configure escalation

Best Practices

- Use group email addresses
- Implement escalation procedures
- Avoid notification overload
- Test notification delivery
- Document on-call procedures

Monitoring Tablespace Usage

Space Metrics

- Used space percentage
- Free space available
- Datafile sizes and counts
- Autoextend status
- Growth rate trends

Alerts for Space Issues

- Warning at 85% full
- Critical at 95% full
- Predictive alerts based on growth
- Datafile maxsize approaching

Space Management Actions

- Add datafiles
- Enable autoextend
- Resize existing datafiles
- Purge unnecessary data
- Compress or archive data

Trainingcred

Session and Connection Monitoring

Active Session Monitoring

- Current session count
- Session activity details
- Long-running sessions
- Blocked sessions
- Resource-intensive sessions

Connection Pool Monitoring

- Connection count trends
- Connection failures
- Pool exhaustion warnings
- Response time metrics

Session Details Available

- Username and program
- SQL currently executing
- Wait events and waits
- Resource consumption
- Session statistics

Trainingcred

Storage and I/O Monitoring

Disk Space Monitoring

- Database file locations
- Free space on file systems
- Growth rate analysis
- Storage utilization trends

I/O Performance

- Read/write operations per second
- I/O response times
- Top I/O files and tablespaces
- I/O distribution across disks

Identifying I/O Bottlenecks

- Hot files (high I/O concentration)
- Slow disk subsystems
- Unbalanced I/O distribution
- Insufficient I/O bandwidth

Database Configuration Management

Configuration Comparison

- Compare initialization parameters
- Identify configuration drift
- Compare across databases
- Track configuration changes

Parameter Management

- View all parameters
- Modify dynamic parameters online
- Schedule parameter changes
- Validate parameter values

Configuration Best Practices

- Standardize across environments
- Document configuration changes
- Version control for configurations
- Regular configuration audits

Job Activity Monitoring

Monitoring Scheduled Jobs

- Job execution status
- Success and failure rates
- Job duration trends
- Resource consumption
- Schedule adherence

Job Management Functions

- Start/stop jobs manually
- Modify job schedules
- View job execution logs
- Manage job dependencies

Troubleshooting Job Issues

- Identify failed jobs
- Review error messages
- Analyze job history
- Adjust job parameters

Trainingcred

Custom Metrics and Dashboards

Creating Custom Metrics

- Define SQL-based metrics
- Set collection frequency
- Establish thresholds
- Configure alerts

Building Custom Dashboards

- Select relevant metrics
- Arrange visual layout
- Add charts and graphs
- Include key performance indicators
- Share with team members

Dashboard Types

- Executive summary dashboards
- DBA operational dashboards
- Application-specific dashboards
- Capacity planning dashboards

SECTION 3: TROUBLESHOOTING COMMON ORACLE DATABASE ISSUES

Troubleshooting Methodology

Systematic Approach

1. Identify and document the problem
2. Gather relevant information
3. Analyze data and logs
4. Develop hypotheses
5. Test potential solutions
6. Implement fix
7. Verify resolution
8. Document for future reference

Information to Collect

- Error messages and codes
- Time of occurrence
- Affected users or applications
- Recent changes
- System resource utilization

Performance Degradation Issues

Common Symptoms

- Slow query response times
- Application timeouts
- High CPU or memory usage
- Excessive I/O waits
- User complaints

Diagnostic Steps

1. Check current system resources
2. Identify long-running sessions
3. Review top SQL statements
4. Analyze execution plans
5. Check for blocking sessions

Quick Checks

Trainingcred
Innovate Advance

- Alert log for errors
- AWR/ADDM reports
- Active session waits
- Database resource usage

Analyzing Slow SQL Queries

Tools for Analysis

- SQL Monitoring reports
- AWR Top SQL section
- V$SQL and V$SQLAREA views
- Execution plan analysis
- SQL Tuning Advisor

Key Metrics to Review

- Elapsed time
- CPU time
- Buffer gets (logical I/O)
- Disk reads (physical I/O)
- Executions count
- Rows processed

Common Problems

- Missing indexes
- Stale statistics
- Inefficient joins
- Full table scans
- Excessive sorting

Execution Plan Analysis

Understanding Execution Plans

- Operations performed by optimizer
- Order of operations
- Access methods (index vs. full scan)
- Join methods (nested loop, hash, merge)
- Cost estimates

Reading Plans

- Operations execute bottom-up, inside-out
- Indentation shows hierarchy
- Cost indicates resource usage
- Cardinality shows row estimates

Identifying Issues

- High cost operations
- Inaccurate cardinality estimates
- Unexpected full table scans
- Inefficient join orders
- Missing or unused indexes

Optimizing Problematic Queries

Optimization Techniques

- Add appropriate indexes
- Update table and index statistics
- Rewrite queries for efficiency
- Use hints judiciously
- Partition large tables

Index Strategies

- Create indexes on WHERE clause columns
- Consider composite indexes
- Use function-based indexes when needed
- Maintain index statistics
- Remove unused indexes

Statistics Management

- Gather fresh statistics
- Use appropriate sample sizes
- Lock statistics for stable data
- Consider histograms for skewed data

Connectivity Problems Overview

Common Connection Errors

- ORA-12154: TNS:could not resolve service name
- ORA-12514: TNS:listener does not know of service
- ORA-12541: TNS:no listener
- ORA-12518: TNS:listener could not hand off
- ORA-12170: TNS:connect timeout

Connection Components

- Client tnsnames.ora configuration
- Listener.ora configuration
- Network connectivity
- Firewall rules
- Database service registration

Trainingcred

Diagnosing TNS Errors

ORA-12154: Service Name Not Resolved

- Check tnsnames.ora file exists
- Verify TNS_ADMIN environment variable
- Confirm service name spelling
- Check file permissions
- Look for syntax errors in tnsnames.ora

ORA-12514: Listener Doesn't Know Service

- Verify service is registered with listener
- Check listener status (lsnrctl status)
- Confirm service_name in database
- Wait for automatic registration (up to 60 sec)
- Force registration (ALTER SYSTEM REGISTER)

ORA-12541: No Listener

- Verify listener is running
- Check listener port (default 1521)
- Confirm hostname resolution
- Test network connectivity (ping, telnet)

Listener Troubleshooting

Checking Listener Status

- Command: lsnrctl status
- Shows listening endpoints
- Lists registered services
- Displays connection statistics

Common Listener Problems

- Listener not started
- Wrong port or protocol
- Hostname not resolvable
- Services not registering
- Connection queue full

Listener Management

- Start: lsnrctl start [listener_name]
- Stop: lsnrctl stop [listener_name]
- Reload: lsnrctl reload
- View config: lsnrctl services

Network Configuration Issues

Testing Network Connectivity

- Ping database host
- Telnet to listener port
- Use tnsping utility
- Check firewall rules
- Verify DNS resolution

Common Network Problems

- Firewall blocking ports
- Network routing issues
- DNS not resolving hostnames
- Network congestion
- Packet loss

Configuration Files

- tnsnames.ora: Client connection definitions
- sqlnet.ora: Client-side parameters
- listener.ora: Listener configuration

Space Management Issues

ORA-01653: Unable to Extend

- Tablespace is full
- Cannot allocate additional extents
- Application transactions may fail

Diagnosis Steps

1. Query tablespace usage
2. Check datafile sizes and limits
3. Identify growing objects
4. Review autoextend settings
5. Check filesystem free space

Immediate Resolution

- Add datafile to tablespace
- Enable autoextend on existing datafiles
- Resize datafiles if space available
- Purge unnecessary data
- Move objects to different tablespace

Tablespace Space Analysis

Key Queries

- DBA_TABLESPACES: Tablespace properties
- DBA_DATA_FILES: File sizes and paths
- DBA_FREE_SPACE: Free space by tablespace
- DBA_SEGMENTS: Object space usage
- DBA_TABLESPACE_USAGE_METRICS: Current usage

Space Usage Metrics

- Total allocated space
- Used space
- Free space
- Percentage used
- Growth rate

Identifying Space Consumers

- Largest tables and indexes
- Tables with high growth
- LOB segments
- Temporary segment usage

Resolving Space Issues

Adding Datafiles

- ALTER TABLESPACE ADD DATAFILE
- Specify size and autoextend
- Consider multiple smaller files
- Distribute across filesystems

Enabling Autoextend

- ALTER DATABASE DATAFILE ... AUTOEXTEND ON
- Set NEXT extent size
- Set MAXSIZE limit
- Monitor growth patterns

Space Reclamation

- Drop unused objects
- Truncate or purge old data
- Shrink segments with excessive free space
- Rebuild fragmented indexes
- Move objects to compressed tablespaces

Temporary Tablespace Issues

Temp Space Problems

- ORA-01652: unable to extend temp segment
- Sorts and hash joins consuming space
- Temporary LOB operations
- Parallel query operations

Monitoring Temp Usage

- V$TEMP_SPACE_HEADER: Total temp space
- V$TEMPSEG_USAGE: Current usage by session
- V$SORT_SEGMENT: Sort segment information
- V$TEMP_EXTENT_POOL: Available extents

Resolution Strategies

- Add tempfiles to temp tablespace
- Increase tempfile size
- Optimize SQL to reduce sorting
- Increase PGA_AGGREGATE_TARGET
- Tune hash join operations

Lock Contention Problems

Symptoms of Lock Contention

- Sessions waiting for locks
- Application timeouts
- "enq:" wait events
- Blocked user transactions
- Transaction throughput drops

Types of Locks

- TM locks: DML table locks
- TX locks: Transaction locks
- UL locks: User-defined locks
- Library cache locks
- Row-level locks

Impact of Lock Contention

- Reduced concurrency
- Poor application performance
- User frustration
- Transaction failures

Identifying Blocking Sessions

Key Views

- V$LOCK: Lock information
- V$SESSION: Session details
- DBA_BLOCKERS: Blocking sessions
- DBA_WAITERS: Waiting sessions
- V$LOCKED_OBJECT: Objects with locks

Finding Blockers

- Query V$LOCK for BLOCK=1
- Join to V$SESSION for details
- Identify blocking SQL
- Determine wait chain
- Calculate wait time

Lock Information to Collect

- Blocking session SID and serial#
- Blocked session details
- Lock type and mode
- Object being locked
- Duration of wait

Resolving Lock Issues

Immediate Actions

- Contact blocking session owner
- Request commit or rollback
- Kill blocking session if necessary
- ALTER SYSTEM KILL SESSION
- Document incident

Preventive Measures

- Minimize transaction duration
- Commit or rollback promptly
- Use appropriate isolation levels
- Implement optimistic locking
- Design for concurrency

Application Design

- Avoid long-running transactions
- Lock in consistent order
- Use row-level locking
- Handle deadlocks gracefully
- Implement retry logic

Archive Log Space Issues

Archivelog Problems

- Archive destination full
- Database hung waiting for archiving
- ORA-00257: archiver error
- Log switches stalled

Symptoms

- Database appears frozen
- Cannot connect new sessions
- DML operations hang
- Alert log shows archive errors

Immediate Resolution

1. Free space in archive destination
2. Delete or move old archive logs
3. Clear alert condition
4. Resume normal operations
5. Implement better management

Diagnostic Tools and Utilities

Alert Log

- Primary diagnostic source
- Records database events
- Shows errors and warnings
- Located in diagnostic_dest/diag/rdbms

Trace Files

- Detailed debugging information
- Generated for specific issues
- Session-level tracing
- Background process traces

Dynamic Performance Views

- V$ views for real-time data
- GV$ views for RAC environments
- System-level statistics
- Session-level information

SQL Scripts

- Custom diagnostic queries
- Performance analysis scripts
- Space management scripts
- Lock detection scripts

Using AWR and ADDM for Troubleshooting

AWR Analysis

- Generate report for problem period
- Compare with baseline period
- Identify top wait events
- Review top SQL statements
- Analyze load profile changes

ADDM Insights

- Automatic performance analysis
- Prioritized findings
- Root cause identification
- Specific recommendations
- Impact quantification

Interpreting Results

- Focus on high-impact findings
- Consider time-based patterns
- Look for sudden changes
- Correlate with known events

Preventive Maintenance Best Practices

Regular Monitoring

- Daily health checks
- Weekly performance reviews
- Monthly capacity planning
- Quarterly architecture reviews

Proactive Actions

- Gather statistics regularly
- Monitor space growth
- Review alert log daily
- Test backup recoveries
- Update documentation

Capacity Planning

- Track growth trends
- Forecast space needs
- Plan for peak loads
- Budget for upgrades
- Scale before problems occur

Trainingcred
Learning Innovate Advance

SECTION 4: APPLYING PATCHES AND UPDATES TO ORACLE DATABASES

Understanding Oracle Patches

What are Patches?

- Software updates from Oracle
- Fix bugs and vulnerabilities
- Improve stability and performance
- Add minor features or enhancements

Why Patch?

- Security vulnerability fixes
- Critical bug corrections
- Performance improvements
- Support requirements
- Compliance mandates

Patch Delivery

- Available through My Oracle Support (MOS)
- Require valid support contract
- Downloaded in zip format
- Include readme files with instructions

Types of Oracle Patches

Release Updates (RU)

- Quarterly cumulative patches
- Include all previous RU fixes
- Recommended for most environments
- Formerly called PSU (Patch Set Updates)

Release Update Revisions (RUR)

- Critical fixes between RUs
- Address urgent issues
- Less common than RUs

One-Off Patches

- Fix specific bugs
- Target individual issues
- Smaller in size
- Applied on top of RU

Security Patches

- Critical Patch Updates (CPU) - deprecated
- Now included in Release Updates
- Address security vulnerabilities
- Should be applied promptly

Bundle Patches

- Exadata-specific patches
- Combine multiple patch types
- Quarterly release schedule

Trainingcred
Learning Unlimited Advance

Trainingcred
Empower. Innovate. Advance.

Oracle Patch Numbering

Understanding Patch Numbers

- Unique identifier for each patch
- Example: 32545013
- Listed in MOS and documentation

Release Update Naming

- Year and quarter: 19.11 (2020 Q4)
- Version and RU number: 19c RU 19.11.0.0.210420
- Release date included

Searching for Patches

- Use patch number directly
- Search by bug number
- Browse by product and version
- Use patch recommendation tool

My Oracle Support (MOS)

Accessing MOS

- Website: support.oracle.com
- Requires Oracle Support account
- CSI (Customer Support Identifier) needed
- Role-based access

Key MOS Features

- Patch downloads
- Knowledge base articles
- Service requests (SR)
- Patch recommendations
- Documentation library

Finding Patches

- Patches & Updates tab
- Search by patch number or description
- Filter by product and platform
- Review patch conflicts

Patch Planning Process

Assessment Phase

1. Review patch readme documentation
2. Check My Oracle Support for known issues
3. Identify prerequisites and conflicts
4. Determine applicability to your environment
5. Review backup requirements

Risk Analysis

- Evaluate patch complexity
- Consider environment criticality
- Review testing requirements
- Assess rollback options
- Plan for contingencies

Stakeholder Communication

- Notify application teams
- Schedule maintenance window
- Communicate expected downtime
- Define success criteria
- Establish communication plan

Patch Prerequisites

Before Applying Patches

- Verify current database version
- Check OPatch version compatibility
- Ensure sufficient disk space
- Verify backup completeness
- Review patch conflicts

OPatch Requirements

- Minimum OPatch version specified in readme
- Update OPatch if necessary
- Download from MOS
- Replace in ORACLE_HOME/OPatch

System Requirements

- Root or sudo access (if needed)
- Oracle software owner permissions
- Network connectivity for downloads
- Adequate temporary space

Testing Strategy

Non-Production Testing

- Apply patch in development environment
- Test in QA environment
- Validate in staging/pre-production
- Never test in production first

Test Scenarios

- Application functionality testing
- Performance regression testing
- Integration testing
- Backup and recovery testing
- Rollback procedure testing

Success Criteria

- All tests pass successfully
- No performance degradation
- No functional issues
- Rollback tested and documented
- Sign-off from stakeholders

Backup Before Patching

Critical Backups Required

1. Full database backup (RMAN)
2. ORACLE_HOME backup
3. Configuration files backup
4. OPatch inventory backup
5. Control file backup

Backup Verification

- Test restore in isolated environment
- Verify backup completeness
- Check backup logs for errors
- Confirm backup location accessibility
- Document backup details

Recovery Plan

Trainingcred

- Document rollback steps
- Identify recovery time objective (RTO)
- Prepare for worst-case scenario
- Have support contacts ready

OPatch Utility Overview

What is OPatch?

- Oracle's patching tool
- Command-line utility
- Manages patch inventory
- Supports patch application and rollback

OPatch Location

- $ORACLE_HOME/OPatch directory
- Must be in PATH
- Version-specific to Oracle release

Key OPatch Commands

- opatch apply: Apply a patch
- opatch rollback: Remove a patch
- opatch lsinventory: List applied patches
- opatch version: Show OPatch version
- opatch prereq: Check prerequisites

Updating OPatch

Why Update OPatch?

- Newer patches require latest OPatch
- Bug fixes in OPatch itself
- Enhanced features
- Compatibility with patch types

Update Steps

1. Download latest OPatch from MOS (Patch 6880880)
2. Backup existing OPatch directory
3. Unzip new OPatch in ORACLE_HOME
4. Verify new version
5. Test with opatch version command

OPatch Version Check

```
opatch version
```

Minimum Version Requirements

- Listed in patch readme
- Specific to database version
- Must meet or exceed minimum

Pre-Patch Verification

Environment Checks

- Set ORACLE_HOME correctly
- Set ORACLE_SID for target database
- Verify oracle user environment
- Check PATH includes OPatch

Database Status

- Ensure database is running
- Check for active sessions
- Review alert log
- Verify no corruption

OPatch Prerequisites Check

```
opatch prereq
CheckConflictAgainstOHWithDetail -ph
./
```

Space Verification

- Check disk space in ORACLE_HOME
- Verify space in /tmp
- Ensure backup destination has space

Patching Steps - Preparation

Download and Extract

1. Download patch from MOS
2. Transfer to database server
3. Extract patch zip file
4. Review README.html or README.txt
5. Note any special instructions

Review Patch Contents

- Patch number and description
- List of bugs fixed
- Prerequisites
- Post-installation steps
- Known issues

Schedule Maintenance

- Choose low-usage time window
- Allow sufficient time (2-4 hours typical)
- Have rollback time available
- Arrange for team availability

Applying Patches - Database Shutdown

Shutdown Considerations

- Coordinate with application teams
- Notify users of downtime
- Stop application connections first
- Shut down cleanly (NORMAL or IMMEDIATE)

Shutdown Steps

1. Stop all applications
2. Close database listeners
3. Shutdown database instances
4. Stop Oracle services (if applicable)
5. Verify all processes stopped

For RAC Environments

- Shut down all instances
- Stop cluster resources
- Coordinate with cluster administrator
- Follow RAC-specific procedures

Applying Patches - Execution

Basic Patch Application

```
cd <patch_directory>
opatch apply
```

OPatch Process

1. Verifies prerequisites
2. Checks for conflicts
3. Backs up files to be modified
4. Applies patch files
5. Updates inventory
6. Reports success or failure

Monitoring Progress

- Review on-screen output
- Watch for errors or warnings
- Note completion time
- Check log files

Common Options

- -silent: Non-interactive mode
- -force: Override warnings
- -invPtrLoc: Specify inventory location

Trainingcred
Learning Unlimited Advance

Applying Patches - Post-Application

Database Startup

1. Start database in UPGRADE mode (if required)
2. Run post-patch SQL scripts
3. Compile invalid objects
4. Restart in NORMAL mode
5. Start listeners

Post-Patch Scripts

- Located in $ORACLE_HOME/rdbms/admin
- datapatch utility (12c and later)
- catbundle.sql (11g)
- Run as SYSDBA

Running datapatch

```
cd $ORACLE_HOME/OPatch
./datapatch -verbose
```

Verification After Patching

Patch Inventory Check

```
opatch lsinventory
```

- Verify patch appears in inventory
- Confirm patch number
- Check application timestamp

Database Registry

```
SELECT * FROM dba_registry_history;
```

- Shows patching history
- Confirms SQL changes applied

Component Status

```
SELECT comp_name, version, status
FROM dba_registry;
```

- Verify all components VALID
- Check version numbers updated

Recompile Invalid Objects

```
@$ORACLE_HOME/rdbms/admin/utlrp.sql
```

Post-Patch Testing

Functional Testing

- Test critical application functions
- Verify database connectivity
- Check scheduled jobs
- Test backup procedures
- Validate integrations

Performance Testing

- Compare key metrics to baseline
- Check query response times
- Monitor resource utilization
- Review execution plans
- Check for regressions

Monitoring Period

- Monitor for 24-48 hours
- Watch alert logs closely
- Check error logs
- Review performance metrics
- Address issues promptly

Rollback Procedures

When to Rollback

- Critical functionality broken
- Severe performance degradation
- Data corruption detected
- Application incompatibility
- Unexpected errors

Rollback Methods

1. OPatch rollback command
2. Restore from backup
3. Clone from standby (if available)

OPatch Rollback

```
opatch rollback -id <patch_number>
```

Restoration from Backup

1. Restore ORACLE_HOME
2. Restore database from backup
3. Recover as needed
4. Verify functionality

Rollback Considerations

- Plan rollback before patching
- Test rollback in non-production
- Document rollback steps
- Have team available during rollback

Trainingcred
Empower. Innovate. Advance.

Patching RAC Environments

RAC Patching Strategies

- Rolling: Minimize downtime
- Non-rolling: All instances down
- Depends on patch type

Rolling Patch Process

1. Patch first node, restart
2. Verify first node healthy
3. Proceed to next node
4. Repeat for all nodes

RAC-Specific Commands

```
opatch auto (for grid infrastructure)
opatchauto (Oracle 12c+)
```

Considerations

- Grid Infrastructure patches first
- Then database home patches
- Use opatchauto for automation
- Follow Oracle documentation exactly
- Extended maintenance window

Patching in Data Guard Environments

Data Guard Patching Strategy

- Patch standby databases first
- Switchover to patched standby
- Patch old primary (now standby)
- Switchback if desired

Benefits of This Approach

- Minimal downtime
- Ability to rollback via switchback
- Test patch on standby first
- Validate before production impact

Steps

1. Patch physical standby
2. Test standby after patch
3. Perform switchover
4. Patch old primary
5. Verify Data Guard configuration

Trainingcred
Learning Unlimited Advance

Automation and Best Practices

Patch Automation Tools

- Oracle Enterprise Manager
- Oracle Fleet Patching and Provisioning
- Custom scripts and frameworks
- Configuration management tools

Benefits of Automation

- Consistent patching process
- Reduced human error
- Faster deployment
- Better documentation
- Audit trail

Best Practices

- Maintain patch calendar
- Follow change management process
- Keep detailed records
- Standardize procedures
- Regular patch reviews

Patch Management Calendar

Oracle's Patch Schedule

- Release Updates: Quarterly (Jan, Apr, Jul, Oct)
- Security patches: As needed
- Bundle patches: Quarterly
- Plan ahead for each release

Internal Schedule

- Review patches: Week 1 after release
- Test in dev: Week 2-3
- Test in QA: Week 3-4
- Apply to production: Week 5-6
- Post-patch monitoring: Ongoing

Planning Considerations

- Business calendars
- Holiday periods
- Peak usage times
- Freeze windows
- Resource availability

Trainingcred
Empower Innovate Advance

Documentation Requirements

Pre-Patch Documentation

- Current database version
- Applied patches inventory
- System configuration
- Backup details
- Maintenance plan

During Patching

- Start and end times
- Commands executed
- Errors encountered
- Resolution steps
- Team members involved

Post-Patch Documentation

- Verification results
- Test outcomes
- Issues and resolutions
- Lessons learned
- Update configuration management database (CMDB)

Common Patching Issues

OPatch Failures

- Insufficient privileges
- Disk space issues
- Conflicting patches
- Inventory corruption

Resolution Steps

- Check OPatch log files
- Verify prerequisites
- Clean up inventory if needed
- Apply prerequisite patches
- Contact Oracle Support if needed

Post-Patch Issues

- Invalid objects after patching
- Application errors
- Performance problems
- Startup failures

Troubleshooting

- Review alert logs
- Check patch logs
- Verify all scripts ran
- Recompile invalid objects
- Restore from backup if necessary

Trainingcred
Empower. Innovate. Advance.

Emergency Patching

When Emergency Patches Needed

- Critical security vulnerability
- Data corruption bug
- Severe performance issue
- Production outage

Accelerated Process

- Compressed testing cycle
- Expedited approvals
- Extended team availability
- Enhanced monitoring

Risk Mitigation

- Extra thorough backups
- Detailed rollback plan
- Senior staff involvement
- Vendor support engaged
- Clear communication

Patch Conflicts and Resolution

Identifying Conflicts

```
opatch prereq
CheckConflictAgainstOHWithDetail -ph
./
```

Types of Conflicts

- File conflicts with existing patches
- Prerequisite patches missing
- Platform incompatibility
- Version mismatches

Resolving Conflicts

- Apply prerequisite patches first
- Use merge patches if available
- Contact Oracle Support
- Review MOS for guidance
- Consider alternative patches

Multi-Tenant Patching (12c+)

CDB Patching Considerations

- Patches applied at CDB level
- Affects all PDBs
- PDBs must be upgraded together
- Extended maintenance window

Patching Steps

1. Patch CDB$ROOT
2. Run datapatch
3. Datapatch updates all PDBs
4. Verify each PDB

PDB-Specific Considerations

- Cannot patch individual PDBs
- All PDBs at same patch level
- Plan for all PDB downtime
- Test representative PDBs

Compliance and Audit

Why Patch Compliance Matters

- Security requirements
- Industry regulations
- Corporate policies
- Support requirements
- Risk management

Tracking Compliance

- Maintain patch inventory
- Document patch levels
- Track patch dates
- Report on compliance status
- Address gaps promptly

Audit Requirements

- Proof of patching
- Change tickets
- Test results
- Approvals
- Risk assessments

Trainingcred
Learner Innovate Advance

Working with Oracle Support

When to Engage Support

- Patch failures
- Unclear documentation
- Conflicting information
- Complex environments
- Critical systems

Service Request Best Practices

- Provide detailed information
- Include error messages
- Attach log files
- Describe environment
- State business impact

Information to Provide

- Patch number attempting
- Database version
- Platform and OS
- OPatch version
- Complete error messages
- Relevant log files

Summary and Key Takeaways

Patching Best Practices

- Plan thoroughly before patching
- Always backup before applying patches
- Test patches in non-production first
- Follow Oracle documentation exactly
- Document everything

Key Success Factors

- Proper testing and validation
- Good communication
- Adequate time allocation
- Skilled team members
- Reliable backups

Ongoing Patch Management

- Stay current with Oracle releases
- Subscribe to security alerts
- Maintain regular patching schedule
- Keep detailed patch history
- Continuously improve processes

Remember

- Patching is essential for security and stability
- Proper planning prevents problems
- Testing is not optional
- Documentation saves time later
- Learn from each patching experience

Trainingcred
Empower. Innovate. Advance.